Assignment: AI-Based Timetable Scheduler

Course: Artificial Intelligence | **Class:** B.Tech, Second Year **Team size:** 3 students | **Duration:** 2 weeks | **Total marks:** 18(+12 bonus) **Submission:** GitHub link, report (PDF), demo.

1. Problem Statement

Every college has to prepare a timetable that places lectures and labs into rooms and time slots without clashes. Build an application that **automatically generates a valid timetable** using the AI search techniques taught in class, and then **compare** those techniques on real data.

2. Learning Objectives

After this assignment you should be able to:

1. Model a real problem as a **Constraint Satisfaction Problem (CSP)** with variables, domains and constraints.
2. Implement **backtracking search** with heuristics (MRV, forward checking, and others).
3. Implement a **local search** method (Min-Conflicts) and explain how it differs from systematic search.
4. Separate **hard constraints** from **soft constraints** and design a cost function.
5. Wrap an AI algorithm inside a usable application.
6. Evaluate and compare algorithms using experiments, and report the results honestly.

3. Input Data

The application must accept data through a form, CSV files or a JSON file. The data includes:

Entity	Fields
Subjects	code, name, weekly hours, type (theory or lab)
Teachers	name, subjects they can teach, unavailable slots, maximum weekly load
Classes/Batches	name, strength, list of subjects, lunch break slot
Rooms	name, capacity, type (classroom or lab)
Time structure	working days, periods per day, break period(s)

Sample datasets (you must create both):

- **Small:** 3 classes, 6 teachers, 4 rooms, 5 days x 6 periods
- **Large:** 8 or more classes, 15 or more teachers, 8 or more rooms, 5 days x 7 periods

Make sure both datasets are actually solvable (check that total required hours do not exceed available room-slots).

4. Constraints

4.1 Hard constraints (must never be violated)

- H1. A teacher cannot teach two sessions at the same time.
- H2. A class cannot have two sessions at the same time.
- H3. A room cannot be used for two sessions at the same time.
- H4. Room capacity must be at least the class strength.
- H5. Lab sessions must be in lab rooms; theory sessions in classrooms.
- H6. A teacher can only be scheduled in their available slots.
- H7. Every subject gets exactly its required weekly hours.
- H8. A teacher's load should not exceed their maximum weekly load.
- H9. Each class must have its lunch break in its assigned time slot, with no session scheduled then.

4.2 Soft constraints (implement at least 3 of the 5)

- S1. Minimize free periods (gaps) in a class's day.
- S2. Spread a subject's lectures across different days (avoid the same subject twice a day, unless it is a lab).
- S3. Lab sessions should occupy consecutive periods (for example, a 2-period lab).
- S4. Respect teacher preferences (for example, no first-period class).
- S5. Avoid more than 2 consecutive lectures for a teacher.

5. Technical Requirements

5.1 Problem formulation (starting hint)

- **Variable:** one lecture session, for example *(Class A, Maths, session 2)*.
- **Domain:** possible *(day, period, room)* combinations. You may fix the teacher in advance (subject-teacher mapping) to keep the domain small.
- **Constraints:** binary constraints between variables that share a teacher, class or room, plus unary constraints (room type, capacity, availability).
- **Cost function for local search:** Cost = 1000 x (hard violations) + (soft penalties)

5.2 Mandatory algorithms

Algorithm 1: Backtracking CSP

- Implement plain backtracking first (this is your baseline).
- Then add **at least three** improvements from this list: MRV, Degree heuristic, Least Constraining Value, Forward checking, AC-3 (as preprocessing).
- Recommended minimum: **MRV + Forward checking + Degree heuristic**.

Algorithm 2: Min-Conflicts local search

- Start from a complete (possibly conflicting) assignment.
- At each step, pick a conflicted variable and move it to the value that minimizes conflicts.
- You must define and justify: the initial assignment, how the conflicted variable is chosen, tie-breaking, and the iteration limit.
- Include a method to escape plateaus (random walk with a small probability or random restarts).

5.3 Optional algorithms (+12 bonus marks)

- **Simulated Annealing:** define neighbourhood moves (swap two sessions, move a session to a free slot), initial temperature, cooling schedule and stopping rule.
- **Hybrid:** use Backtracking to get a feasible timetable, then improve soft-constraint quality with Simulated Annealing.
- **Hill Climbing:** may be included as an extra comparison point.

5.4 Baseline solvers

Python-constraint may be used **only to cross-check your results**, never as your main solution.

5.5 Application requirements

Language: Python with **Streamlit** or **Tkinter** is recommended. The app must have:

1. Data input (form or file upload) with basic validation.
2. An **algorithm selector** (Backtracking, Min-Conflicts, and any optional one).
3. A **Generate** button that shows the time taken and number of violations.
4. Timetable grid views **by class, by teacher and by room**.

5. **Export to CSV or Excel** (PDF is optional).
6. **Manual editing:** the user can move a session to another slot, and the app immediately flags any hard-constraint conflict this creates.

6. Experiments and Comparison (required)

Run both algorithms on **both datasets**. For randomized methods, run **10 times** and report the average.

Metric	Backtracking (baseline)	Backtracking (with heuristics)	Min-Conflicts
Time taken (seconds)			
Nodes expanded / iterations			
Hard violations in final result			
Soft penalty score			
Success rate (out of 10 runs)			

Also include:

- A graph of **cost vs. iterations** for Min-Conflicts (at least one run).
- A short analysis (half a page) answering: *Which method was faster? Which gave better soft-constraint quality? What happens as the problem grows? When would you use each method?*

7. Deliverables

1. **Source code** in a GitHub repository with a README (setup steps, how to run, screenshots).
2. **Both sample datasets** in the repository (JSON or CSV).
3. **Working demo** of the application.
4. **Project report** (6 to 10 pages, PDF) with these sections:
    - Problem formulation (variables, domains, constraints)
    - Algorithm descriptions and design choices
    - Cost function definition
    - Experimental results and comparison
    - Limitations and future work
    - Contribution of each team member
5. **Presentation/demo:** 8 to 10 minutes of demo.

8. Evaluation (18 marks)

Component	Marks
Problem modeling (variables, domains, constraints)	2
Backtracking CSP implementation (heuristics, correctness)	3
Min-Conflicts implementation (correctness, design choices)	2
Quality of timetable (hard constraints met, soft constraints handled)	2
Experiments and comparison analysis	2
Application (usability, views, export, manual edit)	3
Report	2
Presentation	2

Bonus (up to +12):

- Simulated Annealing implemented and compared (+6)
- Hybrid CSP + SA approach (+6)

9. Tips

- Start with a **tiny** example (1 class, 2 teachers) and test your constraints by hand before scaling up.

- Write a separate function that **counts violations** of a timetable. Both algorithms and the manual-edit feature can reuse it.
- Use plain backtracking first. If it is too slow on the large dataset, that is a valid result: report it and show how the heuristics help.
- Keep the algorithm code separate from the UI code.
- Set a random seed so that your experiments are reproducible.

10. Academic Integrity

- You may discuss ideas with other teams, but the code must be your own.
- AI coding assistants may be used, but you must **disclose where** and be able to **explain every part of your code** in the demo. Marks are awarded for understanding, not just working code.
- Cite any paper, tutorial or repository you refer to.
- Teams with very similar code will be called for a separate viva.